

MODULE 2: TASKS IN MEDICAL IMAGE ANALYSIS

HOW TO TRAIN A CNN

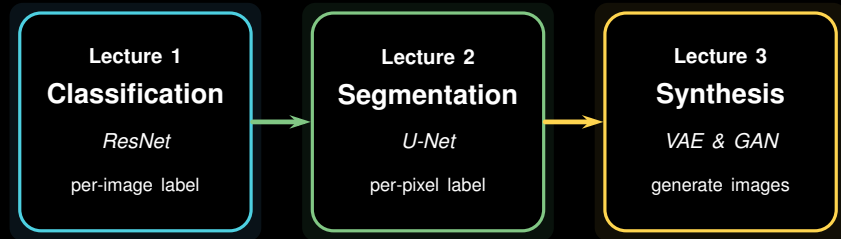
Mehmet Kurt, Tianyi Ren
University of Washington

July 12, 2026

FROM “WHAT IS THE TASK?” TO “HOW DO WE TRAIN IT?”

Module 1 asked **what** tasks exist in medical image analysis.

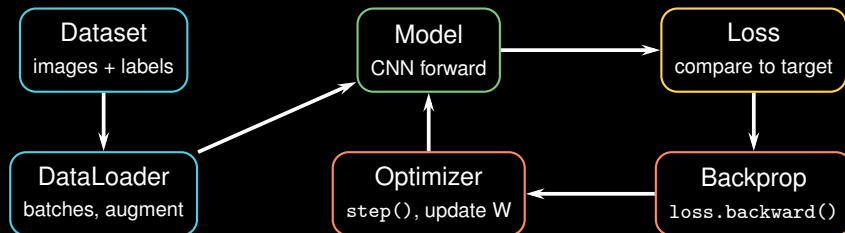
Module 2 asks **how** to actually **train a CNN** for them — one canonical **task**, **architecture**, and **notebook** per lecture.



All three share one recipe — only the model, loss, and metric change.

THE UNIVERSAL TRAINING RECIPE

Every lecture in this module is the same loop — swap in a task-specific **model**, **loss**, and **metric**.



What changes per task

	Classification	Segmentation	Synthesis
Model	ResNet	U-Net	VAE / GAN
Loss	Cross-entropy	Dice / CE	MSE + KL / adversarial
Metric	Accuracy, AUC	Dice, IoU	FID, visual

Part I

LECTURE 1: CLASSIFICATION — RESNET

RECAP: WHERE MODULE 1 LEFT OFF

We ended Module 1 with a classifier that maps an image to a score vector, then to class probabilities:

$$s = f(x_i; W) \quad P(Y=k \mid x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

$$L_i = -\log P(Y=y_i \mid x_i) \quad (\text{cross-entropy})$$

The open question

We defined the **loss**. But *which* function f ?
And how do we **learn** W for a **deep** network?

This lecture

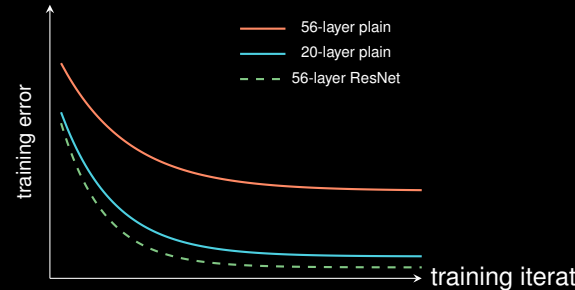
- f = a deep **CNN**
- the block that made depth work: **ResNet**
- train + evaluate it on real medical images

DEEPER SHOULD BE BETTER — BUT PLAIN NETS GET WORSE

- ▶ More layers $\Rightarrow$ richer features (Module 1: edges $\rightarrow$ parts $\rightarrow$ objects).
- ▶ **Degradation:** beyond a point, adding layers *increases* **training** error — not overfitting, an **optimization** problem.
- ▶ A deep plain net struggles to even represent the identity mapping.

The idea

Let each block learn a **residual** correction $F(x)$ on top of its input x , instead of a full new mapping.



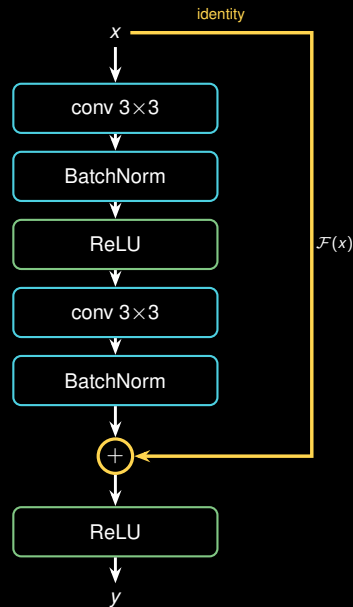
Illustrative — cf. He et al., 2016

THE RESIDUAL BLOCK

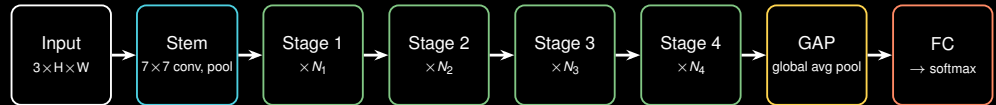
A block computes

$$y = \mathcal{F}(x, W) + x$$

- ▶ The **skip connection** adds the input back.
- ▶ If the ideal map is close to identity, the net just drives $\mathcal{F} \rightarrow 0$ — easy to learn.
- ▶ Gradients flow through the skip $\Rightarrow$ no vanishing in very deep nets.



RESNET: A STACK OF RESIDUAL BLOCKS



Each “stage”

A run of residual blocks at one resolution; the first block halves $H \times W$ and doubles channels.

Depth = block counts

ResNet-18 $N = [2, 2, 2, 2]$

ResNet-34 $N = [3, 4, 6, 3]$

ResNet-50 bottleneck blocks

TRAINING LOOP, CONCRETELY

The universal recipe, in PyTorch:

```
for x, y in loader:
    logits = model(x)
    loss = criterion(logits, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

- ▶ `criterion = CrossEntropyLoss` (softmax + NLL, from Module 1).
- ▶ `optimizer = SGD(+momentum)` or Adam.
- ▶ `zero_grad` → `backward` → `step`: the gradient-descent update to W .
- ▶ One pass over the data = one **epoch**; repeat and watch val accuracy.

We run exactly this in `L1_classification_resnet.ipynb`.

EVALUATING A CLASSIFIER (MEDICAL FRAMING)

Accuracy is not enough

With rare disease (Module 1 long-tail), predicting “healthy” can score 95% and still miss every patient.

Confusion matrix

Per-class right/wrong counts $\Rightarrow$ where the model fails.

Sensitivity / Specificity

Sensitivity = $\frac{TP}{TP + FN}$ (catch disease)

Specificity = $\frac{TN}{TN + FP}$ (avoid false alarms)

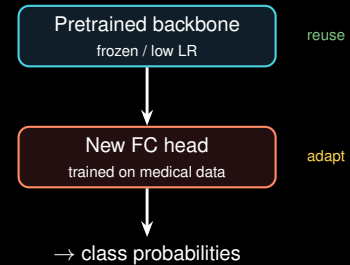
ROC / AUC

Sweep the decision threshold; AUC summarizes the trade-off in one number.

TRANSFER LEARNING: BORROW FEATURES, FINE-TUNE

- ▶ Medical datasets are **small** (Module 1: data scarcity).
- ▶ Start from a ResNet **pretrained on ImageNet** — its early filters (edges, textures) transfer.
- ▶ Replace the final FC layer with one for *your* classes.
- ▶ **Fine-tune**: train the new head (and optionally later stages) on the medical data.

Often the difference between overfitting and a usable model.



HANDS-ON: RESNET ON PATHMNIST

Notebook

L1_classification_resnet.ipynb

1. `pip install medmnist`; pick GPU runtime.
2. Load **PathMNIST** (9-class digital pathology, RGB 28×28).
3. ResNet-18: from-scratch *and* pretrained + fine-tune.
4. Train (CrossEntropy + Adam), track loss/accuracy.
5. Evaluate: accuracy, confusion matrix, ROC/AUC.

→ Free Colab GPU is enough; a 1-epoch smoke test runs in <1 min.

[PathMNIST sample montage
exported by the notebook]

PathMNIST samples (exported by the notebook).

MedMNIST v2 — Yang et al., Scientific Data 2023.

PITFALLS & WHAT TO EXPECT

Overfitting

Train acc $\gg$ val acc. Fixes: augmentation, weight decay, early stopping, pre-trained weights.

Leakage

Same patient in train *and* test inflates accuracy. Split by **patient**, not by image.

Class imbalance

Long-tailed classes get ignored. Fixes: class weights, resampling, focal loss; watch *per-class* metrics.

Expect

Fine-tuned ResNet-18 on PathMNIST reaches high-80s to 90s% test accuracy in a few epochs.

Part II

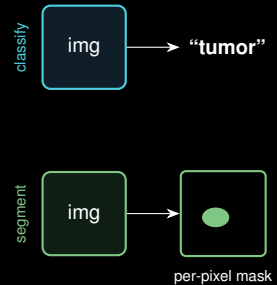
LECTURE 2: SEGMENTATION — U-NET

CLASSIFICATION → SEGMENTATION

- ▶ Classification: **one label per image** (“tumor present”).
- ▶ Segmentation: **one label per pixel** (“*which* pixels are tumor”).
- ▶ Output is a **mask** the size of the image — delineation, volume, boundaries (Module 1 segmentation task).

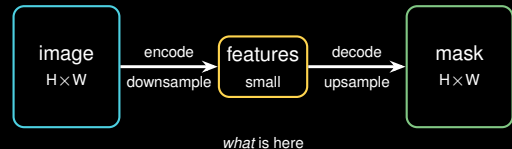
Consequence

We must *recover* spatial resolution after the CNN downsamples — an **encoder–decoder**.

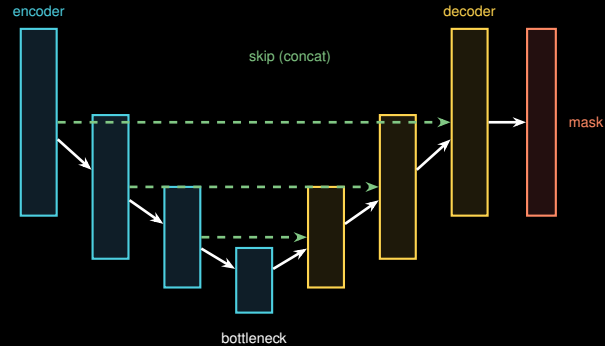


THE RESOLUTION PROBLEM

- ▶ A CNN **downsamples** to build semantic features — great for *what*, but it throws away *where*.
- ▶ Segmentation needs a **full-resolution** output.
- ▶ Solution: downsample to understand, then **upsample** back to pixel space — an **encoder-decoder**.



THE U-NET



Contracting path

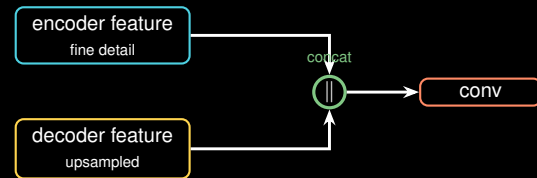
Conv + downsample: *what* is in the image (context).

Expanding path + skips

Upsample and **concatenate** encoder features: *where* it is (localization).

UPSAMPLING & SKIP CONNECTIONS

- ▶ **Upsample** the coarse decoder feature (transposed conv or interpolation) to grow $H \times W$ back.
- ▶ **Concatenate** the matching encoder feature — it still holds the *fine spatial detail* lost to pooling.
- ▶ Result: semantics *and* precise boundaries; skips also ease gradient flow.



LOSS: CROSS-ENTROPY VS. DICE

Pixelwise cross-entropy

Treat every pixel as its own classification. Simple — but a **tiny lesion** is swamped by background, so the net can score well by predicting “all background”.

In practice

Combine both: $\mathcal{L} = \text{BCE} + \text{Dice}$.

Dice loss

Overlap-based, robust to imbalance:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|P \cap G|}{|P| + |G|}$$

P = prediction, G = ground truth. Small foreground still dominates the score.

METRICS: DICE & IoU

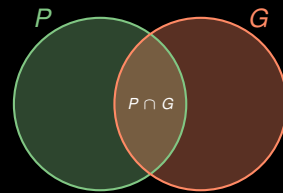
Overlap metrics

$$\text{Dice} = \frac{2|P \cap G|}{|P| + |G|} \quad \text{IoU} = \frac{|P \cap G|}{|P \cup G|}$$

Both in $[0, 1]$; higher is better.

Boundary error

Hausdorff distance catches the *worst* boundary miss — overlap metrics can look fine while an edge is badly off.



prediction P vs. ground truth G

TRAINING LOOP, CONCRETELY

Same recipe as Lecture 1 — only the **model**, **loss**, and **output** change:

```
for x, mask in loader:
    logits = unet(x)
    loss = bce(logits, mask)
        + dice_loss(logits, mask)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
# predict: sigmoid(logits) > 0.5
```

- ▶ Model outputs a **per-pixel** logit map, not a class vector.
- ▶ sigmoid → threshold at 0.5 gives the binary mask.
- ▶ Track **Dice** on validation (not just loss).
- ▶ Heavy **augmentation** (flips, rotations, elastic) — masks are scarce.

HANDS-ON: U-NET ON KVASIR-SEG

Notebook

L2_segmentation_unet.ipynb

1. Download **Kvasir-SEG** (polyp images + masks, ~46 MB).
2. Resize to 128×128 ; train/val split.
3. U-Net from scratch (encoder-decoder + skips).
4. Train with BCE + Dice; track **val Dice**.
5. Overlay predicted vs. ground-truth masks.

[image | GT mask | prediction
exported by the notebook]

Kvasir-SEG overlay (exported by the notebook).

→ Free Colab GPU; QUICK_RUN smoke test runs
in ~1 min.

PITFALLS & WHAT TO EXPECT

Class imbalance

Tiny foreground $\Rightarrow$ plain CE collapses to background. Use **Dice** (or combined) loss.

Small data

Few annotated masks $\Rightarrow$ heavy augmentation; patient-level split to avoid leakage.

Boundaries

Overlap can look good while edges leak. Watch **Hausdorff**; consider boundary loss.

Expect

A from-scratch U-Net reaches **Dice** $\approx$ **0.8–0.9** on Kvasir polyps within a few epochs.

Part III

LECTURE 3: SYNTHESIS — VAE & GAN

WHY SYNTHESIZE MEDICAL IMAGES?

- ▶ **Augmentation:** more training data, especially **rare cases** (Module 1 long-tail).
- ▶ **Modality translation:** e.g. CT $\leftrightarrow$ MRI.
- ▶ **Denoising / super-resolution** (Module 1 preprocessing task).
- ▶ **Privacy:** share *synthetic* images instead of patient data.

Goal

Learn the data distribution $p(x)$ so we can **sample** new, realistic images.

GENERATIVE MODELING

Discriminative

Learn $p(y \mid x)$ — a decision boundary (Lectures 1–2).

Generative

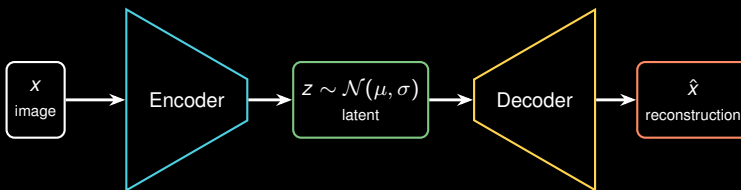
Learn $p(x)$ — the *data itself*, so we can draw new samples $x \sim p(x)$.

Two families we cover

- ▶ **VAE** — latent-variable, likelihood-based; stable.
- ▶ **GAN** — implicit, adversarial; sharp.

Both map a simple noise/latent z to an image.

VAE: ENCODE $\rightarrow$ LATENT $\rightarrow$ DECODE



Encoder

Compresses x into a *distribution* over a low-dim latent z (outputs μ, σ).

Decoder

Reconstructs an image from z . To **generate**, sample $z \sim \mathcal{N}(0, I)$ and decode.

VAE: THE LOSS (ELBO)

Two terms

$$\underbrace{\|x - \hat{x}\|^2}_{\text{recon (MSE)}} + \underbrace{\text{KL}(q(z|x) \parallel \mathcal{N}(0, I))}_{\text{regularize latent}}$$

- ▶ Recon: $\hat{x}$ must look like x .
- ▶ KL: keep the latent close to $\mathcal{N}(0, I)$ so sampling is meaningful.

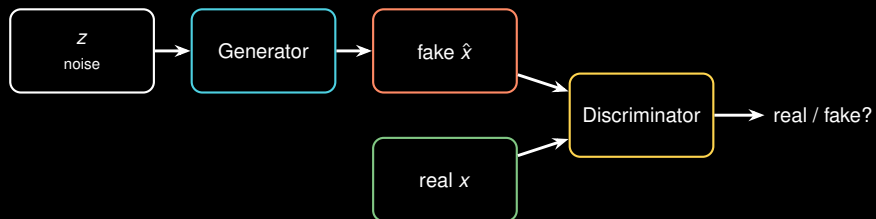
Reparameterization

Sampling isn't differentiable — rewrite it:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Now gradients flow through μ, σ and we can backprop.

GAN: GENERATOR VS. DISCRIMINATOR



Generator G

Maps noise z to an image; never sees real data directly — only D 's feedback.

Discriminator D

A binary classifier: real vs. fake. Its gradient teaches G .

GAN: THE ADVERSARIAL GAME

Minimax objective

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

D maximizes (catch fakes); G minimizes (fool D).

```
# alternate each step
train D: real→1, fake→0
train G: D(fake)→1 # fool D
```

It's unstable

- ▶ **Mode collapse:** G outputs one “safe” image.
- ▶ D too strong $\Rightarrow G$'s gradient vanishes.

Tricks: balanced LR, label smoothing, normalization, WGAN-GP.

VAE vs. GAN

	VAE	GAN
Training	stable	unstable (collapse)
Samples	blurry	sharp
Objective	likelihood (ELBO)	adversarial (implicit)
Latent	explicit, structured	implicit
Good for	smooth interpolation, anomaly	realistic augmentation

Modern hybrids (VAE-GAN, diffusion) blend the two — Module 3.

HANDS-ON: VAE & GAN ON MEDMNIST

Notebook

L3_synthesis_vae_gan.ipynb

1. Load a MedMNIST set (28–32 px).
2. Build + train a **VAE** (MSE + KL); sample a grid.
3. Build + train a small **DCGAN**; sample a grid.
4. Compare VAE (blurry) vs. GAN (sharp).

VAE runs on free Colab; GAN is GPU-heavy → Colab Pro.

[generated sample grid
exported by the notebook]

Synthetic samples (exported by the notebook).

EVALUATION & PITFALLS

Evaluation is hard

- ▶ **FID:** distance between real/fake feature distributions.
- ▶ **Downstream:** does synthetic data improve a real task?
- ▶ Open problem (Module 4: evaluation challenges).

Pitfalls

- ▶ VAE: *posterior collapse* (ignores z) $\rightarrow$ blur.
- ▶ GAN: *mode collapse*, non-convergence.
- ▶ **Hallucination:** synthetic anatomy can mislead — never diagnose on it.

Next: Module 3 — beyond task-specific models (self-supervision, diffusion, foundation models).